

ATmega32A Module

PWM (Pulse Width Modulation)

Ali Sahafi

1 Theory

PWM produces an “analog-like” output from a digital pin by switching it on and off very fast. The fraction of time the pin is HIGH is the **duty cycle**: at 25 % duty an LED appears dim, at 100 % fully bright. The switching is far too fast for the eye (or a motor) to notice — only the *average* voltage matters:

$$V_{\text{avg}} = V_{\text{CC}} \cdot \frac{\text{duty}}{255}$$

In **Fast PWM** mode the timer counts 0–255 continuously; the output pin is HIGH while the counter is below the compare register and LOW above it. Writing the compare register therefore *is* setting the duty cycle. With the driver’s fixed prescaler of 8:

$$f_{\text{PWM}} = \frac{F_{\text{CPU}}}{8 \cdot 256} \approx 3.9 \text{ kHz at } 8 \text{ MHz}$$

PWM uses dedicated output pins — the compare hardware drives them directly:

Channel	Pin	Timer	Duty register
OC0	PB3	Timer0	OCR0
OC1A	PD5	Timer1	OCR1A
OC1B	PD4	Timer1	OCR1B

2 API reference

Method	Description
<code>PWM.initTimer0_FastPWM()</code>	Fast-PWM on PB3 (pin set to output automatically)
<code>PWM.setDutyCycleTimer0(duty)</code>	Set Timer0 duty cycle, 0–255
<code>PWM.initTimer1_FastPWM_8bit()</code>	Fast-PWM on PD4 (OC1B) and PD5 (OC1A)
<code>PWM.setDutyCycleTimer1A(duty)</code>	Set PD5 duty cycle, 0–255
<code>PWM.setDutyCycleTimer1B(duty)</code>	Set PD4 duty cycle, 0–255

Warning: Timer conflict. PWM occupies the timer hardware, so the CTC functions of Module 3 and the PWM functions must never share a timer:

- `Timer.initTimer0()` conflicts with `PWM.initTimer0_FastPWM()`
- `Timer.initTimer1()` conflicts with `PWM.initTimer1_FastPWM_8bit()`

Both write the same TCCR registers, and the later call silently breaks the earlier one. Timer2 remains free for CTC interrupts alongside PWM.

3 Example: breathing LED

Wiring.

- LED + 330 Ω resistor from **PB3** (OC0) to GND.

Build & flash. `make pwm`

```
/*
 * PWM Example -- breathe (fade) an LED on PB3 using Timer0 Fast PWM
 *
 * Wiring:
 * - LED + 330R resistor from PB3 (OC0) to GND
 *
 * Build & flash: make pwm
 */

#define F_CPU 8000000UL
#include "pwm.hpp"

int main() {
    PWM.initTimer0_FastPWM(); // PB3 is configured as output automatically

    while (true) {
        // Fade up
        for (uint16_t duty = 0; duty <= 255; duty++) {
            PWM.setDutyCycleTimer0(duty);
            _delay_ms(4);
        }
        // Fade down
        for (int16_t duty = 255; duty >= 0; duty--) {
            PWM.setDutyCycleTimer0(duty);
            _delay_ms(4);
        }
    }

    return 0;
}
```

Note the loop variables: counting *up* uses `uint16_t` and counting *down* uses a signed `int16_t` — an 8-bit `uint8_t` could never exit either loop. Why?

4 Exercises

1. The brightness change looks “front-loaded”: most of the visible change happens at low duty values. Why? (Hint: the human eye responds logarithmically.) Try stepping through a table of perceptually-spaced duty values instead.
2. Use `initTimer1_FastPWM_8bit()` to fade two LEDs on PD4 and PD5 in *opposite* directions (one brightens while the other dims).
3. Connect the potentiometer from the ADC module (Module 5) and set the LED brightness from the potentiometer position: 10-bit ADC value $\rightarrow$ 8-bit duty cycle.